

Lecture Notes

Design & Analysis of Algorithm

Brijesh Bhatt

December 15, 2020

(The following notes briefly describe the topics covered in the classroom. Students need to refer to the textbooks and related material for detailed description.)

Chapter 1

Greedy Method

(Lecture 1, 2, 3)

1.1 Optimization Problem

Optimization problems are those problems in which our aim is to minimize or maximize some cost function. The cost function that we try to maximize or minimize is known as **objective function**. The process of maximization or minimization often involves a range of *choices*. The goal of optimization is to *select the best possible choice* under the given *criteria*. Various possible choices for a solution are referred as **feasible solutions**. Amongst the feasible solutions, the one which maximizes/minimizes the objective function is called an **optimal solution**. When maximization/minimization is required to be achieved under some constraint, the problem is called **Constrained Optimization**. (Optimization without any constraint is known as ‘unconstrained optimization’).

Formally the optimization problem can be define as follows,

“Given a function $f : A \rightarrow R$ from some set A to the real numbers and a function $g : A \rightarrow \{0, 1\}$, search an element x_0 in A such that $f(x_0) \geq f(x)$ for all $x \in A$ and $g(x_0) = 1$.”

In mathematical formulation this is referred as $\operatorname{argmax}_x f(x)$, which means ‘those values of x for which $f(x)$ is maximum’.

- *Objective function* : The function which we aim to maximize. In above definition, $f(x)$ is the objective function.
- *Constraint* : The condition that x must satisfy. In above definition, $g(x)$ is the constraint.

- *Feasible Solution* : All solutions for which $g(x_0) = 1$.
- *Optimal Solution* : A feasible solution which maximizes $f(x)$, $\operatorname{argmax}_x f(x)$

1.1.1 Combinatorial optimization

An optimization problem with discrete variables is known as a combinatorial optimization problem. In a combinatorial optimization problem, we are looking for a solution from a finite (or possibly countable infinite) set of feasible solutions. In other words, a set of feasible solutions is discrete and our aim is to find the best solution.

A brute-force approach to solve any combinatorial optimization problem is to enumerate all feasible solutions and then select the best amongst them. For a given input set S of size n , generating all possible subsets of feasible solution takes $O(2^n)$ time (Why?). Thus, for a large input it is very difficult to generate all solutions and find the optimal solution.

1.2 Knapsack Problem

Knapsack problem, a combinatorial optimization problem, is defined as follows,

‘Given a set of items I of size n , where each item $i \in I$ has an associated profit P_i and weight W_i ; select a subset I' , $I' \subseteq I$, for which $\operatorname{argmax}_x \sum_{i \in I'} P_i$ and $\sum_{i \in I'} W_i \leq C$. Here C is a constant which represents total capacity of the knapsack’.

Every subset of I for which $\sum_{i \in I'} W_i \leq C$, are feasible solutions of the problem. Our aim is to find out a subset for which $\sum_{i \in I'} P_i$ is maximum. If we try to enumerate all feasible solution (or try to pick a feasible solution randomly and check for optimality) it can take $O(2^n)$

Knapsack problem has two variants, *0/1 knapsack* and *fraction knapsack*. In fractional knapsack, a fraction of item is allowed to be placed in the knapsack (in final solution). In 0/1 knapsack either an item is fully included (1) or completely ignored (0). Consider an instance of knapsack problem shown in table 1.1. If the capacity (C) of knapsack is 5, then the set of feasible solutions is as follows,

$$F = \{\{i_1\}, \{i_2\}, \{i_3\}, \{i_4\}, \{i_2, i_3\}\} \quad (1.1)$$

The optimal solution which gives maximum profit, is $\{i_2, i_3\}$.

Item	i_1	i_2	i_3	i_4
Profit	10	6	8	12
Weight	5	2	3	5

Table 1.1: knapsack example instance

Item	i_1	i_2	i_3	i_4	i_5	i_6	i_7
Profit	10	6	8	12	7	5	3
Weight	5	2	3	5	4	3	1

Table 1.2: Exercise: knapsack example

0/1 knapsack is a *hard problem* as it is difficult to find optimal solution in polynomial time (more on this later). There are strategies which provide good approximation for the problem. In the following sections we will discuss greedy method and try to solve knapsack problem using greedy method.

Exercises

Exercise 1.2.1 (Knapsack example) *How many feasible solutions are there for the knapsack instance shown in table 1.2? What is the optimal solution for fractional knapsack? What is the optimal solution for 0/1 knapsack? (Capacity of the knapsack is 10.)*

Exercise 1.2.2 (Complexity of brute force) *Derive complexity of naive approach of enumerating all feasible solution and find out the optimal solution.*

1.3 Greedy Method

Greedy method tries to generate optimal solution without exploring all feasible solutions. A greedy algorithm for optimization problem usually starts with an empty solution set and with each iteration generates a *partial solution* by adding one element into the set. It is necessary to ensure that the (partial) solution generated after each iteration is a feasible solution. A *selection function* needs to be defined to add an item during each iteration. Greedy method ensures that the *selection* made in each iteration is the best choice. General structure of a greedy method is as shown in algorithm 1.

Basic underlying assumption in the greedy method is, ‘If the subsolution at each iteration is optimal, then the final solution is also optimal’. However, greedy method does not always guarantee an optimal solution. Greedy method is popular because it is often the most simple and intuitive approach.

Algorithm 1: Greedy Method (general outline)

Data: $a[1:n]$ contains n inputs
1 solution = Φ ;
2 **for** $i \leftarrow 1$ **to** n **do**
3 $x[i] = \text{select}(\text{next best candidate from the set } a[1 : n])$;
4 **if** $\text{Feasible}(x[i], \text{solution})$ **then**
5 $\text{solution} = \text{solution} \cup x[i]$;
6 **return** solution ;

Algorithm 2: Greedy algorithm for (fraction) knapsack problem

Data: $I[1:n]$ item array, $p[1:n]$ profit, $w[1:n]$ weight, C Knapsack Capacity, $s[1:n]$ solution vector
1 $I[1:n] = \text{Sort}(I[1:n], p[1:n], w[1:n])$;
2 **for** $i \leftarrow 1$ **to** n **do**
3 $s[I[i]] = 0$;
4 **for** $i \leftarrow 1$ **to** n **do**
5 **if** $w[I[i]] \leq C$ **then**
6 $s[I[i]] = 1$;
7 $C = C - w[I[i]]$;
8 $P = P + p[I[i]]$;
9 **else**
10 $s[I[i]] = C/w[I[i]]$;
11 $C = 0$;
12 $P = P + s[I[i]] * p[I[i]]$;
13 **Print** Total Profit P ;
14 **return** s ;

1.4 Greedy solution for knapsack problem

In knapsack problem, our aim is to maximize the profit of the items selected while keeping the overall weight of the items less than the knapsack capacity.

Item	i_1	i_2	i_3	i_4
Profit	10	6	8	12
Weight	5	2	3	5
PWRatio	2	3	2.66	2.4

Table 1.3: knapsack example: PW Ratio

Item	i_2	i_3	i_4	i_1
------	-------	-------	-------	-------

Table 1.4: knapsack example: Sorted Itemset

So, an intuitive way is to select items which gives maximum profit with minimum weight. This results in a simple and very efficient algorithm to solve knapsack problem. The basic process is as follows, first sort all items in descending order of *profit/weight* ratio, then put items, one by one in that order, into the knapsack until you reach the maximum capacity. Algorithm 2 describes the complete algorithm.

Let's try to understand working of the algorithm using the knapsack instance described in table 1.1. As shown in the table, $n = 4$, as there are total 4 items. Let's say, the capacity of the knapsack is 7. In first step of the algorithm, the data is sorted by P/W ratio. Table 1.3 describes P/W ratio of each item. After sorting the item set in descending order of P/W ratio, the sorted item set would be as shown in the table 1.4. $s[1 : n]$ is a binary array which stores solution of the knapsack problem. It is encoded as shown below¹,

$$s[i] = \begin{cases} 1, & \text{if item } i \text{ is in the knapsack} \\ 0, & \text{if item } i \text{ is not in the knapsack} \\ c/w[i], & \text{if fraction of item } i \text{ is in the knapsack} \end{cases}$$

As shown in the step 2 and 3 of the Algorithm 2, s is initialized with all zero values, e.g. no item is selected in the knapsack. After that (from step 4 onwards) the algorithm iterates over the sorted item set and adds the item in the knapsack if the weight of the item is less than the available knapsack capacity. During an iteration, if an item, i , is added in the knapsack (step 6, 7, 8), $s[i]$ is set to 1, profit $P[i]$ is added to total profit, and knapsack capacity is reduced by $W[i]$, weight of the item. Look at the iteration 1 and 2, shown in the table 1.5.

¹Fractional value of $s[i]$ is applicable only for fractional knapsack problem

Iteration	Item	$C - W_i$	$P + P_i$	s_i	knapsack
1	i_2	$7 - 2$	$0 + 6$	$s_2 = 1$	$\{i_2\}$
2	i_3	$5 - 3$	$6 + 8$	$s_3 = 1$	$\{i_2, i_3\}$
3	i_4	0	$14 + (2/5)12$	$s_4 = (2/5)$	$\{i_2, i_3, (2/5)i_4\}$

Table 1.5: knapsack example: iterations of the algorithm

In the third iteration of the algorithm, the remaining capacity of the knapsack is 2 and the next best item is i_4 . Since the weight of i_4 is more than the remaining capacity, the condition defined in the step 5 will be false and the algorithm will execute step 9, 10, 11, 12². Since the total available capacity is 2, only $2/5$ of the item can be updated in the knapsack. Hence, $2/5$ of the profit is added to the total profit, as shown in the row three of the table 1.5.

Step 5 to 8 are repeated until the weight of the current item is less than the available knapsack capacity. When weight of the current item is more than the capacity, $C/W[i]$ fraction of the item is added to the knapsack to fully fill the knapsack. After addition of this item no more items can be added to the knapsack, as the capacity of the knapsack is fully utilized. A fraction of the profit is added to the total profit. The total profit thus found is the optimal value for the given instance.

Algorithm 2 is greedy in the sense that it generates optimal solution directly (Without exploring all feasible solutions). In order to do this, it uses sorting as a selection function to select items. With each iteration the best element is added in the solution set. It assumes that if the best element is selected in each iteration then the final solution will be optimal. This algorithm works for fraction knapsack, however it does not guarantee the optimal solution for 0/1 knapsack problem.

Exercises

Exercise 1.4.1 (Greedy algorithm) *Describe working of algo 2 for the knapsack instance defined in table 1.2 (assume capacity $C=10$). Show item selected and weight, profit updation for each iteration. Also, give the final solution. (For both, fraction and 0/1 knapsack)*

Exercise 1.4.2 (Greedy 0/1 knapsack) *Give an instance of 0/1 knapsack for which algo 2 does not give optimal solution.*

²step 9, 10, 11, 12 are required only for fractional knapsack problem

Exercise 1.4.3 (Making change Problem) *Making change problem is defined as follows, ‘Given an infinite supply of coins of different denominations and an amount to be changed; find out minimum number of coins which sums equal to the total amount’. For example, Let’s say we have coins of Rs. 1, 2, 3, and 5 and we want change of 10 Rs. Then possible solutions are {5, 5} (Two coins of 5 Rs), {5, 3, 2} (One coin of 5, 3, and 2 each), {2, 2, 2, 2, 2} (Five coins of 2 Rs), etc. The optimal solution is ‘Two coins of 5 Rs’. Describe a Greedy algorithm to solve making change problem. Does the proposed Greedy algorithm solve the problem optimally? If yes, then prove. If no, then give an example for which Greedy does not give optimal solution of the problem.*

1.5 References and further reading

1. Chapter 4, Greedy Method, Fundamentals of Computer Algorithms, Horowitz, Sahni and Rajasekaran
2. Chapter 6, Greedy algorithms, Fundamentals of Algorithm, G. Brassard and P. Bratley, Pearson